



Instituto Tecnológico de Puerto Rico Recinto de San Juan

DOCUMENTACIÓN DEL PROGRAMA PERSONAL

Versiones 1.0, 2.0 y 3.0

Plantilla para acompañar el Laboratorio 3 - Semana 4

Estudiante	Yadiel J. River Ojeda
Repositorio	TESI2105_Programa_Evolutivo_ApellidoNombre
Nombre del programa	Registro de Horas de Trabajo
Fecha	

1. DESCRIPCIÓN GENERAL Y CONTINUIDAD

Describe brevemente la necesidad que atiende tu programa y explica cómo las versiones anteriores preparan el punto de partida para v3.0. No repitas las respuestas del documento del laboratorio; resume la continuidad en 4 a 6 oraciones.

El programa Registro de Horas de Trabajo atiende la necesidad de calcular de forma rápida y sin errores el pago semanal de un empleado que trabaja por hora. Lo utilizaría un empleado o un supervisor que necesita registrar cuántas horas se trabajaron cada día de la semana y obtener el pago total correspondiente, incluyendo el pago de tiempo extra cuando el total de horas supera las 40 horas regulares. El programa comenzó como una calculadora sencilla de un solo cálculo (v1.0) y evolucionó para registrar horas día por día y aplicar reglas de tiempo extra (v2.0), manteniendo siempre la misma idea central.

2. RESUMEN DE LAS VERSIONES ANTERIORES

Versión	Base técnica	Cómo se refleja en mi programa
v1.0	Programa secuencial con entradas, variables, constantes, operadores y salidas.	Calcula la paga por hora al registrar la cantidad de horas trabajadas y la cantidad de paga por hora, luego restando ese resultado con una tarifa
v2.0	El mismo programa incorpora decisiones y ciclos.	Calcula las horas por día individualmente por un periodo de 5 días. Luego calcula si las horas trabajadas pasan un límite de 40 horas para darle paga adicional.

Este resumen no sustituye la documentación entregada en la Semana 3; únicamente establece continuidad para la nueva versión.

3. VERSIÓN 3.0 - ORGANIZACIÓN MEDIANTE FUNCIONES

Explica en 3 a 5 oraciones cómo reorganizaste el código de v2.0 para crear v3.0 mediante funciones con responsabilidades claras.

Para crear v3.0, el código de v2.0 se dividió en cinco funciones con responsabilidades separadas: pedir los datos del empleado, registrar las horas de la semana, calcular el pago, mostrar los resultados y organizar el flujo principal. Cada bloque de código que antes estaba escrito de forma lineal ahora vive dentro de una función con un nombre que describe claramente su tarea. La decisión y el ciclo que ya existían en v2.0 se conservaron sin cambios en su lógica, solo se movieron dentro de las funciones correspondientes. El resultado es un programa que hace exactamente lo mismo que v2.0, pero organizado de una manera más clara y fácil de seguir.

4. FUNCIONES CREADAS EN V3.0

Completa una fila por cada función creada. Debes documentar al menos tres funciones.

Función	Propósito	Parámetros	Retorno o acción
Pedir_datos_empleados()	Solicitar al usuario el nombre del empleado y el pago por hora.	Ninguno	Retorna: nombre, pago_por_hora
registrar_horas_semana(dias_trabajados)	Pedir mediante un ciclo while las horas trabajadas de cada día y sumarlas.	Días_trabajados	Retorna: total_horas
Main()	Organizar y llamar en orden las funciones.	ninguno	Accion: ejecuta el flujo completo; no devuelve valor

5. FUNCIÓN INCORPORADA DE PYTHON

Función incorporada (built-in)	Round()
Dónde se utiliza	Dentro de main(), en la línea pago_total = round(pago_total, 2), justo después de llamar a calcular_pago().
Propósito dentro del programa	Redondear el pago total a dos decimales antes de mostrarlo, para presentar una cantidad de dinero clara y realista en vez de una cifra con decimales excesivos.

6. INTEGRACIÓN DE LA LÓGICA DE V2.0

Decisión que se conserva	La estructura if/elif/else que determina si el total de horas trabajadas superó las 40 horas regulares y calcula el pago de tiempo extra a 1.5 veces la tarifa.
Función donde se encuentra la decisión	calcular_pago(total_horas, pago_por_hora)
Ciclo que se conserva	El ciclo while que pide las horas trabajadas de cada uno de los 5 días de la semana.
Función o parte del flujo donde se ejecuta el ciclo	registrar_horas_semana(dias_trabajados)

7. PRUEBAS DE FUNCIONAMIENTO DE V3.0

Registra al menos dos pruebas realizadas después de modularizar el programa.

Prueba	Datos utilizados	Resultado esperado	Resultado obtenido
1	Nombre: Juan Pérez. Pago por hora: \$10. Horas por día: 8, 8, 8, 8, 6 (total 38 horas).	No debe activarse tiempo extra; pago total = 38 x \$10 = \$380.00.	El programa mostró el mensaje de que no se alcanzaron las 40 horas regulares y calculó \$380.00, tal como se esperaba.
2	Nombre: María Ortiz. Pago por hora: \$10. Horas por día: 9, 9, 9, 9, 9 (total 45 horas).	Deben activarse 5 horas extra a 1.5 veces la tarifa; pago total = (40 x \$10) + (5 x \$10 x 1.5) = \$475.00.	El programa identificó 5 horas extra y calculó un pago total de \$475.00, tal como se esperaba.
3	Nombre: Carlos Díaz.	No debe activarse tiempo	El programa mostró el

Prueba	Datos utilizados	Resultado esperado	Resultado obtenido
	Pago por hora: \$12. Horas por día: 8, 8, 8, 8 (total 40 horas exactas).	extra por ser exactamente 40 horas; pago total = 40 x \$12 = \$480.00.	mensaje de semana regular completa sin tiempo extra y calculó \$480.00, tal como se esperaba.

8. REFLEXIÓN DE EVOLUCIÓN

¿Por qué v3.0 es más fácil de leer, reutilizar o mantener que v2.0? Explica en 4 a 6 oraciones y menciona al menos una función de tu programa como ejemplo.

La versión 3.0 es más fácil de leer porque cada función tiene un nombre que indica exactamente qué hace, así que basta con leer main() para entender el flujo completo del programa sin revisar todos los detalles internos. También es más fácil de mantener, porque si en el futuro cambia, por ejemplo, la forma de calcular el tiempo extra, solo hay que modificar la función calcular_pago() sin tocar el resto del programa. Por último, es más reutilizable, ya que funciones como registrar_horas_semana() podrían usarse en otra parte del programa o en un programa distinto simplemente llamándolas con los parámetros adecuados, algo que no era posible cuando toda la lógica de v2.0 estaba escrita de una sola vez sin dividir en partes.

9. VALIDACIÓN

Etiquetas verificadas en el repositorio: X v1.0 X v2.0 X v3.0

Archivo PDF en documentacion/: Documentacion_V1_V2_V3.pdf

URL del repositorio: https://github.com/RivoYad114/TESI2105_Programa_Evolutivo_Yadiel_Rivera.git

Nota: Esta plantilla no requiere diagramas de flujo ni diagramas modulares. La evidencia se concentra en la organización funcional del código, su documentación y las pruebas realizadas.